

Sesiunea 2: Integrarea API-urilor OpenAI în n8n

Capitolul 4: Manipularea Datelor, Maparea Dinamică și "Execute Command".....	1
1. Descărcarea Fișierului: Introducere în "Maparea Dinamică".....	1
2. Deblocarea Nodului "Execute Command".....	4
3. Executarea Comenzilor în Sistemul de Operare.....	5
4. Ieșirile sistemului (Exit Codes).....	7
5. Gestionarea Erorilor (Error Routing).....	8
✓ Soluții și Răspunsuri pentru Trainer.....	9

Capitolul 4: Manipularea Datelor, Maparea Dinamică și "Execute Command"

📝 În acest capitol, facem tranziția de la "ascultare" la "acțiune". În lumea DevOps și QA, nu este suficient să știm că a apărut un document nou; trebuie să îl procesăm. Vom învăța cum să pasăm date de la un nod la altul (Mapare Dinamică) și cum să folosim cel mai puternic nod din n8n pentru a executa comenzi direct în sistemul de operare.

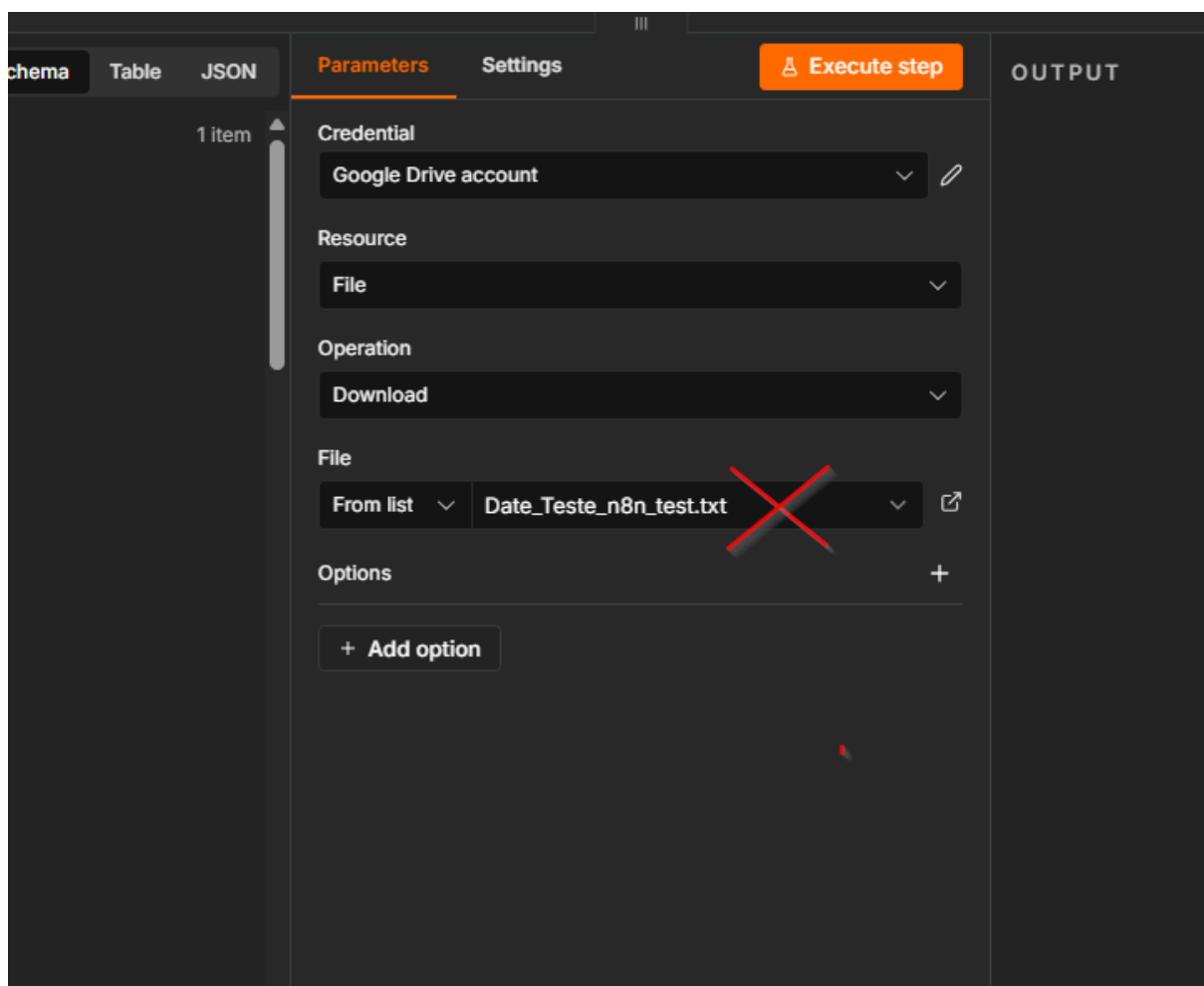
Până acum, primul nostru nod (Trigger-ul) a detectat că ai încărcat un PDF sau un fișier text (ex: plan de testare) în folderul tău din Google Drive. Dar nodul de tip Trigger doar *detectează* evenimentul și expune datele despre el (nume, format, ID), nu descarcă efectiv fișierul pe hard disk-ul tău.

1. Descărcarea Fișierului: Introducere în "Maparea Dinamică"

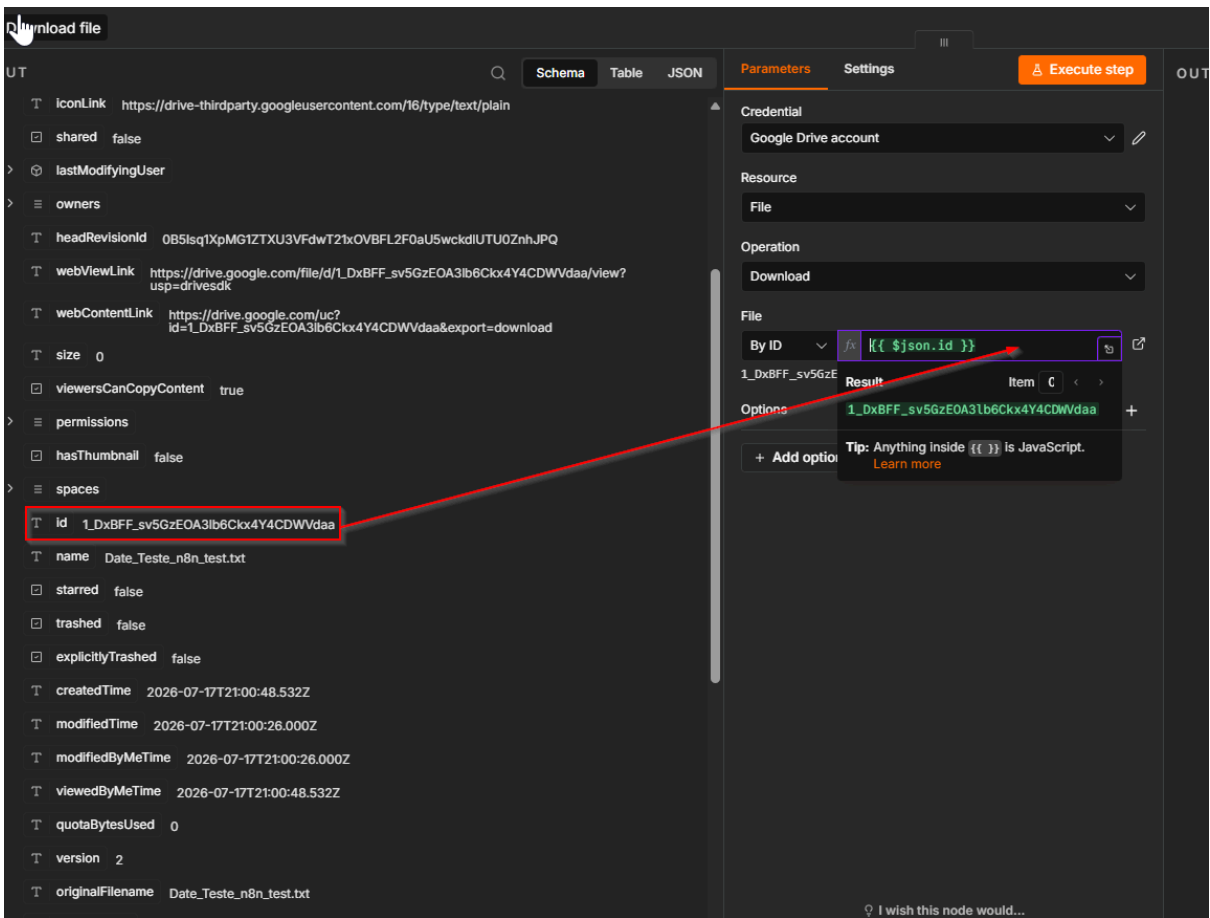
Pentru a aduce documentul pe mașina noastră locală, vom adăuga un al doilea "braț robotic" în flux.

- Pe canvas, apasă pe micul semn + (sau trage o linie din punctul verde) din dreapta nodului Trigger creat anterior. Caută și adaugă aplicația **Google Drive**.

- În panoul din dreapta, secțiunea **Parameters**, vom configura exact acțiunea dorită. Asigură-te că setările sunt următoarele:
 - **Resource:** Alege **File**
 - **Operation:** Alege **Download**
- Aici intervine o capcană pentru începători la câmpul **File**. Implicit, n8n îți oferă opțiunea *From list* pentru a alege un fișier cu mâna (cum se vede în captura noastră). *Atenție! Ca ingineri de automatizare, nu vom selecta niciodată un fișier fix din acea listă ("hardcodare"). Dacă am face asta, la fiecare rulare viitoare, nodul ar descărca la nesfârșit același fișier vechi, ignorând documentele noi.*



- **Maparea Dinamică (Drag & Drop):** Uită-te în panoul din stânga ecranului, denumit **INPUT**. Acolo vezi toate datele expuse de Trigger-ul anterior (cum ar fi mimeType, parents, etc.). Derulează până găsești câmpul id (ID-ul unic al fișierului nou). Prinde acest câmp cu mouse-ul și trage-l (Drag & Drop) direct peste câmpul **File** din panoul drept, înlocuind selecția manuală.

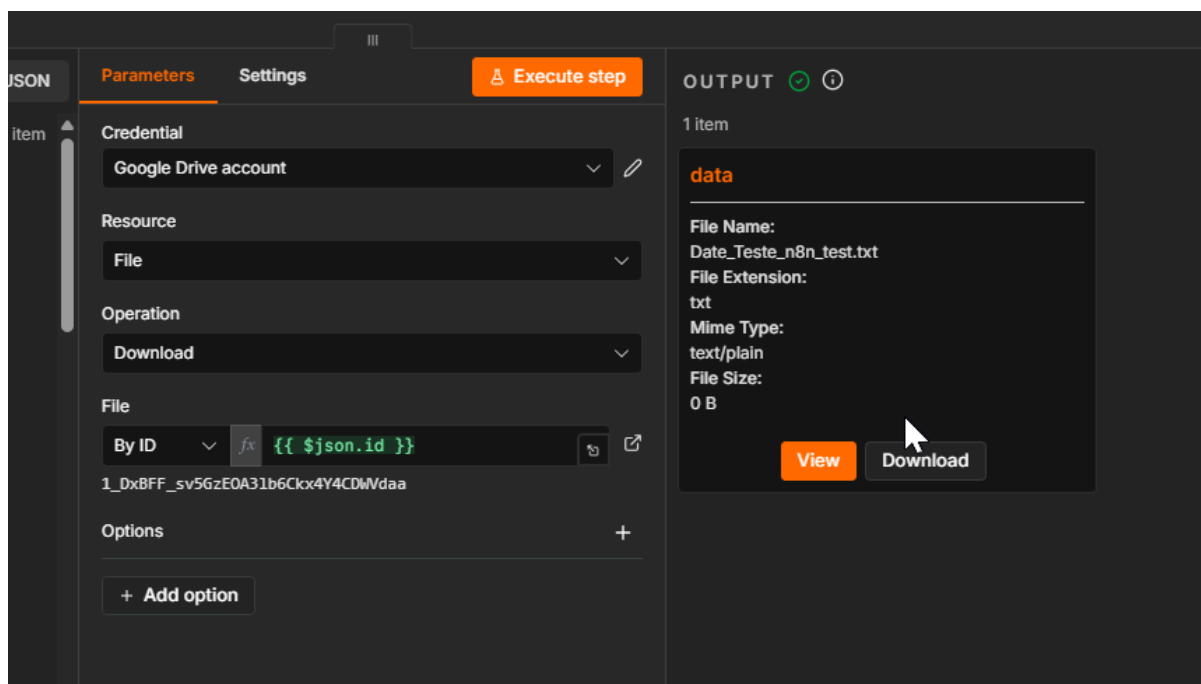


The screenshot shows the n8n workflow editor. On the left, a JSON object represents a Google Drive file. The 'id' field is highlighted with a red box. On the right, the 'Download file' step is configured. The 'File' section is set to 'By ID', and the 'ID' field contains the dynamic expression `{{ $json.id }}`. A red arrow points from the highlighted 'id' in the JSON to the expression field. Below the configuration, a 'Result' table shows the value of the expression as `1_DxBFF_sv5GzEOA3lb6Ckx4Y4CDWVdaa`.

IconLink	https://drive-thirdparty.googleusercontent.com/16/type/text/plain
shared	false
lastModifyingUser	
owners	
headRevisionId	0B5isq1XpMG1ZTXU3VFdwT21xOVbFL2F0aU5wckdIU0ZnhJPQ
webViewLink	https://drive.google.com/file/d/1_DxBFF_sv5GzEOA3lb6Ckx4Y4CDWVdaa/view?usp=drivesdk
webContentLink	https://drive.google.com/uc?id=1_DxBFF_sv5GzEOA3lb6Ckx4Y4CDWVdaa&export=download
size	0
viewersCanCopyContent	true
permissions	
hasThumbnail	false
spaces	
id	1_DxBFF_sv5GzEOA3lb6Ckx4Y4CDWVdaa
name	Date_Test_n8n_test.txt
starred	false
trashed	false
explicitlyTrashed	false
createdTime	2026-07-17T21:00:48.532Z
modifiedTime	2026-07-17T21:00:26.000Z
modifiedByMeTime	2026-07-17T21:00:26.000Z
viewedByMeTime	2026-07-17T21:00:48.532Z
quotaBytesUsed	0
version	2
originalFilename	Date_Test_n8n_test.txt

- n8n va genera o expresie dinamică de tipul `{{ $json.id }}`. Aceasta îi spune platformei: *"La fiecare rulare, uită-te la evenimentul curent și descarcă fișierul detectat fix în cea secundă"*.

Apasă butonul portocaliu **Execute step** (vizibil sus sau în dreapta). Fișierul tău a fost descărcat temporar în memoria platformei n8n!



2. Deblocarea Nodului "Execute Command"

Pentru a scrie log-uri sau a rula scripturi de test pe calculatorul tău, n8n are un nod suprem: **Execute Command**. Acesta îți permite să rulezi comenzi de Terminal (Bash, CMD, PowerShell) direct din interfața vizuală.

De ce nu îl găsești la căutare în n8n?

Începând cu versiunea 2.0, din motive stricte de securitate (pentru a preveni ștergerea accidentală a fișierelor de sistem), n8n a ascuns acest nod din instalările standard. Pentru a-l folosi, trebuie să îi spunem platformei că știm ce facem, devenind adevărați SysAdmins.

Cum îl activăm (Troubleshooting de tip DevOps):

- Mergi în Terminalul tău (unde rulează n8n în fundal) și oprește serverul apăsând **Ctrl + C**.
- Trebuie să setăm o nouă Variabilă de Mediu numită **NODES_EXCLUDE** (care implicit blochează noduri periculoase) la o valoare goală `[]`. Rulați comenzile de mai jos (nu uitați să re-aplicați și variabila de securitate SSL din Capitolul 2):
- **Pe Mac / Linux:**

```
export N8N_SECURE_COOKIE=false
```

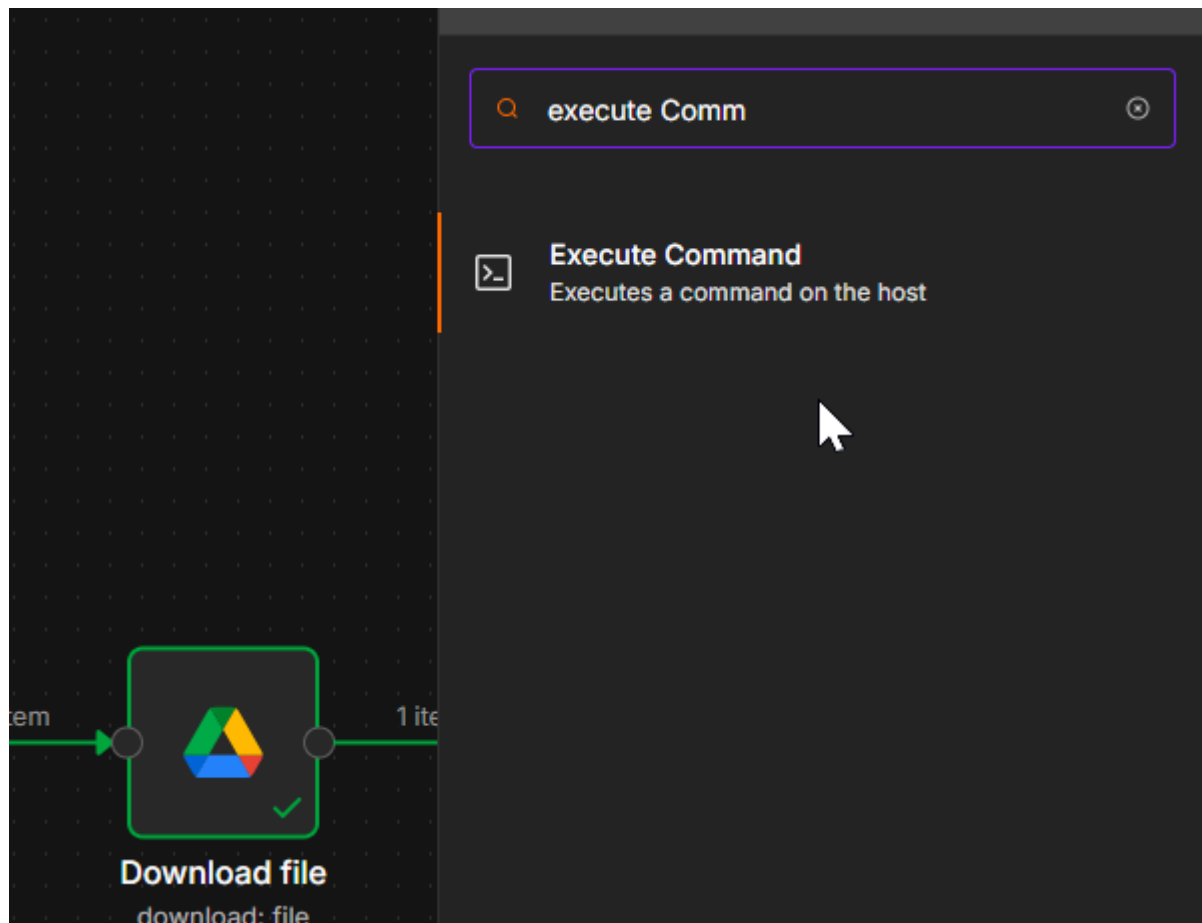
```
export NODES_EXCLUDE="[]"
```

```
n8n
```

- **Pe Windows (PowerShell):**

```
$env:N8N_SECURE_COOKIE="false"
$env:NODES_EXCLUDE="[]"
n8n
```

Odată ce n8n a repornit, dă un refresh la pagina din browser. Dacă adaugi un nou pas, nodul **Execute Command** este acum deblocat și gata de acțiune!



3. Executarea Comenzilor în Sistemul de Operare

Platforma n8n are acum acces complet la mașina ta. Vom demonstra acest lucru creând un registru (Log File) direct pe hard disk în care vom nota numele fiecărui plan de testare

descărcat.

- Apasă din nou pe semnul + din dreapta nodului de descărcare (Download File) și caută nodul proaspăt deblocat, **Execute Command**.
- În câmpul **Command**, scrie începutul comenzii tale de terminal: `echo "A fost descărcat fisierul:`
- Acum, folosește din nou panoul din stânga (Input) pentru a trage parametrul `name` (numele fișierului original) direct în interiorul comenzii tale, după spațiu.
- Continuă comanda închizând ghilimelele și adăugând operatorul de scriere într-un fișier text: `" >> n8n_logs.txt`
- Expresia ta finală va arăta fix așa:

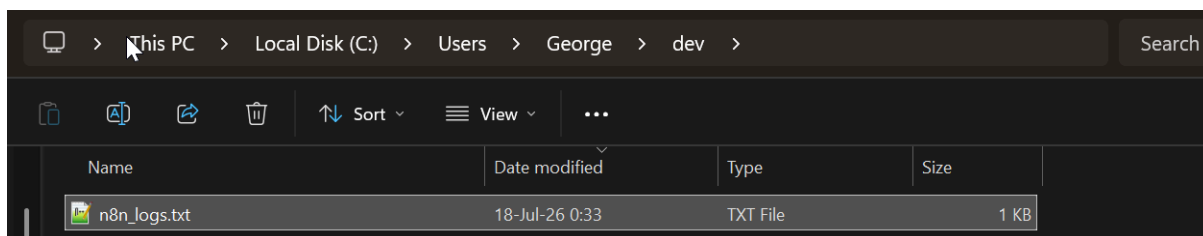
```
echo "A fost descărcat fisierul: {{ $json.name }}" >> n8n_logs.txt
```

💡 Capcană de DevOps: Unde s-a salvat fișierul?

Dacă ai apăsat **Execute step**, n8n a rulat comanda silențios. Dar unde este fișierul `n8n_logs.txt`?

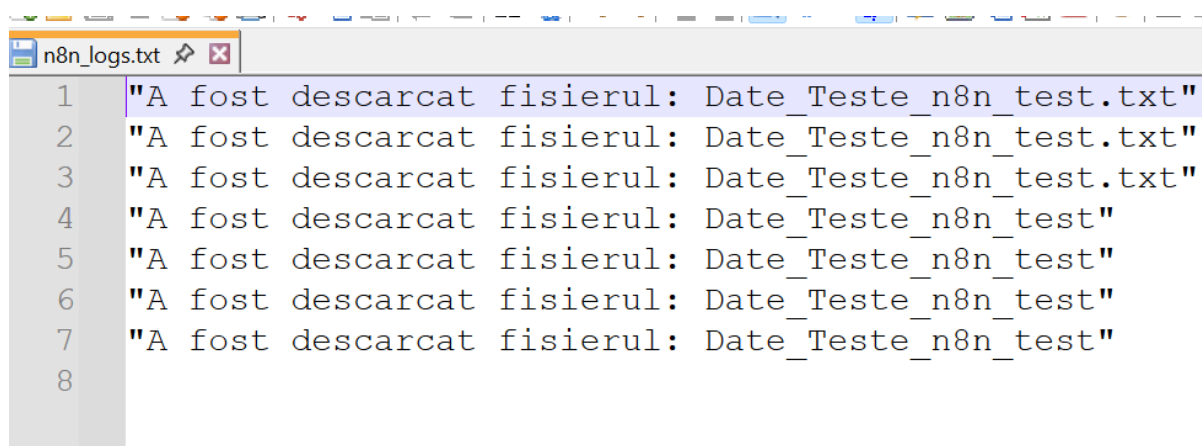
Acesta s-a salvat în *Current Working Directory (CWD)*. Adică exact în folderul în care era deschis Terminalul tău atunci când ai pornit serverul cu comanda `n8n`.

Vrei să afli calea exactă? Șterge temporar comanda cu echo și rulează comanda `cd` (pe Windows) sau `pwd` (pe Mac). În panoul de Output vei vedea printată adresa exactă a folderului unde se află log-ul tău!



💡 Ochi de Arhitect (Data Flow): Unde sunt log-urile mele?!

Dacă deschizi fișierul `n8n_logs.txt` pe care tocmai l-ai găsit, vei observa ceva ciudat: el conține doar textul pe care l-am scris noi ("A fost descărcat fisierul..."). Unde sunt miile de linii de text din interiorul documentului original pe care l-ai descărcat? Răspunsul este esențial pentru a înțelege n8n: Comanda noastră `echo` a folosit doar parametrul `$json.name`, adică o "etichetă" (Metadata). Noi nu i-am spus sistemului să citească fișierul. Dar stai liniștit, fișierul descărcat nu s-a pierdut! Nodul Google Drive a luat tot acel conținut masiv și l-a stocat în siguranță direct în **Memoria RAM a platformei n8n (Date Binare)**. Platforma ține informația "pe țeavă", ascunsă de hard disk, fiind perfect pregătită să o trimită Inteligenței Artificiale în capitolul următor!



4. Ieșirile sistemului (Exit Codes)

După execuția nodului de comandă, în panoul de rezultate din dreapta vei observa un câmp tehnic vital: `exitCode`.

În informatică, orice comandă executată pe un sistem de operare raportează un status la finalizare:

- **Exit Code 0:** Înseamnă succes total. Comanda a rulat fără nicio eroare.
- **Exit Code 1 (sau orice alt număr mai mare ca 0):** Înseamnă că a apărut o eroare (ex: lipsă permisiuni, folder inexistent, sintaxă greșită, teste picate).

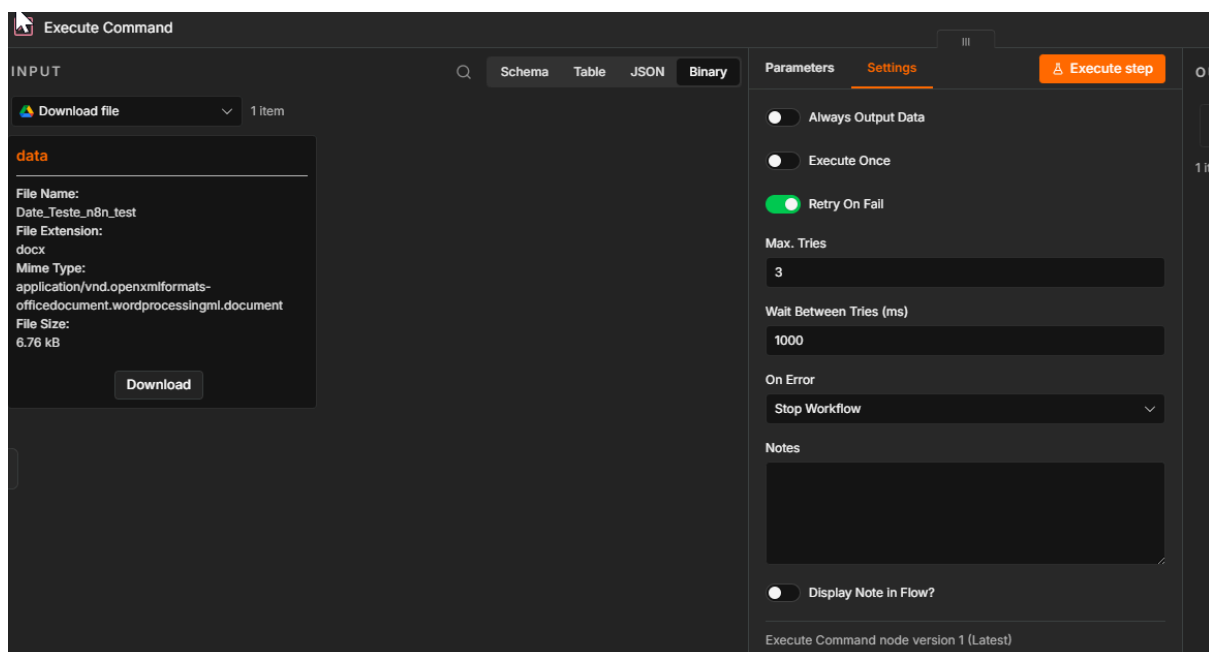
Acest exitCode va deveni parametrul nostru cel mai important în capitolele următoare. Îl vom trimite către Inteligența Artificială pentru a decide dacă trebuie să alarmeze echipa.

5. Gestionarea Erorilor (Error Routing)

Să trecem la un scenariu de DevOps pur. Ce se întâmplă dacă un script pică, terminalul returnează o eroare de sistem, iar nodul "Execute Command" eșuează?

Implicit, n8n oprește întregul flux și îl marchează cu roșu. Dar noi vrem să fim proactivi, vrem alerte!

- În fereastra oricărui nod (inclusiv Execute Command), dă click pe tab-ul **Settings** (lângă Parameters).
- Găsește opțiunea **Continue On Fail** (Continuă la eșec) și activează-o.
- Acum, chiar dacă nodul eșuează, fluxul merge mai departe, iar în datele de ieșire JSON va apărea un câmp special numit error. Acest câmp devine vital pentru a crea alerte inteligente în viitor.



💡 Știați că...?

💡 Capabilitățile nodului "Execute Command" pot fi extinse dincolo de serverul local? Folosind nodul geamăn numit **SSH**, poți orchestra execuții pe mașini aflate la distanță. Imaginați-vă că fișierul descărcat este un script de configurare; prin **n8n** și **SSH**, puteți rula comenzi care să construiască automat o întreagă infrastructură nouă de testare în cloud!

🧱 Poveste Aplicată: "Păpușarul și Marioneta"

- **Scripting-ul tradițional:** Este ca și cum tu ești pe o scenă și miști manual fiecare membru al marionetei (sistemul de operare) tastând linie cu linie. Dacă te oprești din tastat, marioneta se oprește.
- **Execute Command în n8n:** n8n devine *Păpușarul* din spatele cortinei. El are firele deja legate de marionetă. Tot ce face este să aștepte un semnal din exterior (Trigger-ul) și trage automat de fire, executând comanda exact când și cum a fost instruit, în timp ce tu, ca inginer, ești în public, doar supervizând spectacolul.

🕵️ Exercițiu de Gândire: "Polițistul Rutier"

Tocmai ai configurat un nod *Execute Command* care pornește o suită de teste automate Selenium pe un server local. Comanda s-a terminat, iar panoul de output îți returnează `exitCode: 130`.

Sarcina ta: Fără să scrii cod, cum ai configura vizual următorul pas în n8n pentru a te asigura că echipa ta primește un email de alertă *doar* dacă testele pică, și nu este deranjată absolut deloc dacă testele trec cu succes (exit code 0)?

✅ Soluții și Răspunsuri pentru Trainer

Soluție Exercițiul "Polițistul Rutier":

Pentru a evalua rezultatul și a lua o decizie bazată exclusiv pe `exitCode`, trebuie să folosim un nod logic de rutare.

- Imediat după nodul *Execute Command*, vom adăuga un nod de tip **IF** (sau *Switch*).
- Vom configura condiția din interiorul nodului IF astfel încât să extragă valoarea dinamică `{{ $json.exitCode }}`.

- Regula logică va fi: **IF** exitCode este *Egal cu* (Equal) 0 -> Trimite fluxul pe ramura de **True**. Dacă este diferit (ex: 130) -> Trimite fluxul pe ramura de **False**.
- Vom lega un nod de *Send Email* sau *Slack* **DOAR** pe ieșirea ramurii de **False**.
Astfel, dacă codul este 0, fluxul se oprește silențios. Dacă apare o eroare, fluxul o ia pe ruta de eșec și declanșează alerta!